

本プロジェクトの全検討プロセス、要求仕様、思考アルゴリズム（プロンプト）、および実装アーキテクチャを体系的にまとめた開発仕様・設計リファレンスドキュメントです。

後から読み返した際に、全体の流れとポイントがひと目で把握でき、必要に応じてコードや設定の再現ができるよう、凝縮したボリューム（要点を押さえた総合ドキュメント）としてまとめています。

写真分析・講評自動化パイプライン 開発仕様・設計リファレンス

（プロジェクト名: PhotoCritic AI / analyze_photos.py）

第1章 プロジェクト概要と解決課題

1.1 開発背景と目的

カメラ愛好家（撮影者）が撮影した大量の写真（月間数十～百枚規模）に対し、単なる技術データの記録にとどまらず、「写真観の向上」「表現上のハッとする気づき」「フォトブック等の作品集選定の補助」を提供する自動AI講評パイプラインの構築を目的とする。

1.2 解決された主要課題

1. 処理の自動化とOS連携: Macの「写真」アプリと連携し、手動での画像アップロードや整理の手間をゼロ化。
2. AI出力の「金太郎飴化（定型化）」打破: 初期AIモデルで見られた「三分割法」「柔らかい光」「季節ポエム」などの安易なテンプレ表現を、プロンプトの思考手順改修により完全排除。
3. 視覚情報とメタデータの整合: 撮影時刻ファクト（朝日/夕日など）と実際の画像描写（強烈な直射光や眩しさ）を正しく統合。
4. 堅牢な実行基盤: 通信ハング、タイムアウト、OpenAIの確率的セーフティ拒否、トークンレートリミット（429 Error）への完全な自動リカバリ対応。

第2章 要求仕様（Requirements Specification）

2.1 インプット仕様

- 対象画像: macOS「写真」アプリ内の指定アルバム（01_AI解析用）に格納された画像。
- EXIFデータ: カメラ名、レンズ名、撮影日時、F値、シャッタースピード、ISO感度、焦点距離、露出補正值、撮影モード。
- 撮影者の意図・悩み（キャプション）: 写真アプリ上の「情報（説明）」欄に入力された自由記述テキスト（※空欄時は「指定なし」として動作）。

2.2 アウトプット仕様

1. **Markdownノート**: ~/写真分析ノート/YYYY/ 傘下に、撮影年別フォルダで保存。
2. **年間集約ログ（RAG / NotebookLM用）**: ~/Googleドライブ/写真分析ログ_YYYY.txt に全講評テキスト（Markdown）を蓄積。

3. 写真アプリへの書き込み & 整理:

- メタデータ更新: 写真の「タイトル」にAI生成タイトル、「説明」に講評本文を自動挿入。
- アルバム振分け: 処理完了写真を O2_解析完了 アルバムへ自動追加(O1_AI解析用 からの重複処理を防止)。

2.3 重複判定ルール

- 判定基準:「ファイル名のみ(拡張子含む)」で判定。
- 動作: 保存先フォルダ(~/写真分析ノート/YYYY/)に同名の .md ファイルが存在する場合、更新日時に関わらず SKIP(スキップ)処理。これにより、アルバム移動やコピーによるタイムスタンプ変動で不要な枝番(_v2.md)が生成されるのを防止。

第3章 プロンプト & 思考アルゴリズム設計

3.1 思考プロセスの歴史と改善プロセス

- 初期課題: EXIFテキストを先にAIに与えると、画像を細部まで観察せずテキスト(撮影日時や設定)から安易なボエムや定型講評を生成する傾向があった(アンカリング効果)。
- 到達した解決策 (Visual-First アプローチ):
 1. **Step 1(純粹視覚観察)**: テキスト情報を見ずに、ピクセル情報(主副関係、明暗、陰影、視線誘導、質感、空気感)のみを純粹に観察・言語化させる。
 2. **Step 2(EXIF照合・補正)**: 最後にEXIFと照合し、「なぜその描写になったか」の答え合わせ・技術評価を行う。

3.2 評価トーンとプロンプト厳格ルール

- ゴールデンバランス(50 : 50):
 - 称賛 50%: 撮影者の視点、意図、情緒的価値、美点を深く肯定・言語化。
 - 批評 50%: 撮影者の悩みに対する回答、構図や背景処理の課題、次回現場で使える具体的数値(「一歩寄る」「F8まで絞る」等)の提示。
- 光の解釈ルール:
 - 「朝 = 柔らかい光」という固定観念を完全禁止。
 - 撮影時刻(04:00~08:00=朝日 / 16:00~19:00=夕日)を絶対ファクトとしつつ、直射光の眩しさや鋭いハレーション、暗く引き締まった影は「現実の強さ」としてありのまま描写させる。

3.3 出力フォーマット構造

Markdown

■TITLE: [写真の本質を射抜くタイトル(15文字以内)]

■SUMMARY: [写真全体を表す魅力的キャッチコピー(25文字以内)]

■SCORES:

- ・構図・構成: ★★★★★☆ (4/5)
- ・光・色彩: ★★★★★★ (5/5)
- ・ストーリー: ★★★★★☆ (4/5)
- ・技術・露出: ★★★★★☆ (3/5)
- ・独自・世界観: ★★★★★☆ (4/5)

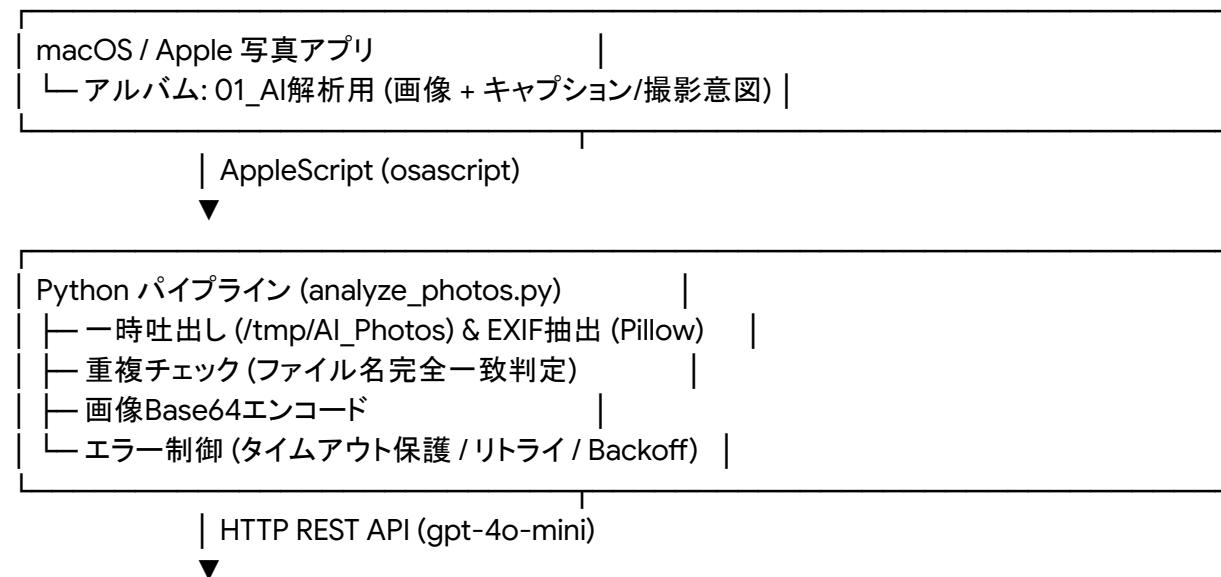
ファイル名 / 撮影日時 / カメラ / レンズ / 設定 / 撮影意図

- ##【1. 情景・空気感とストーリー性】
- ##【2. 視線誘導と構成の美学】
- ##【3. 光の強弱・色彩と印象解析】
- ##【4. EXIFデータの技術的役割と表現効果】
- ##【5. 撮影者のためのステップアップ・アドバイス】
- ##【6. フォトブック&SNSでの役割提案】
- ##【7. 自動タグ】

第4章 機能設計とパイプラインアーキテクチャ

4.1 全体システム構成図

Plaintext



```
OpenAI API (Vision Model) |
└─ Visual-First 思考 & 50:50評価プロンプト実行 |
```

| 講評テキスト返却



```
アウトプット自動整理 (マルチデスティネーション) |
└─ 1. Markdown保存 (~写真分析ノート/YYYY/*md) |
└─ 2. 年間ログ追記 (~Googleドライブ/写真分析ログ_YYYY.txt)
└─ 3. 写真アプリ書込 (タイトル・説明更新 & 02_解析完了へ移動)|
```

4.2 異常系・リカバリ設計(エラーハンドリング)

1. スリープ対策: caffeinate -i -s -w [PID] により、実行中のMacのスリープを完全防止。
2. 写真抽出タイムアウト: iCloud同期遅延等を考慮し、AppleScript実行タイムアウトを **300秒(5分)** に設定。
3. **API通信ハング防止**: OpenAIリクエストに **90秒**のタイムアウトを設定。
4. 確率的セーフティ拒否・通信エラーへの自動リトライ:
 - 判定閾値の境界でAI安全フィルターに弾かれた場合、3秒待機して 最大**2回**(計**3回**)自動再試行。
5. **429 Rate Limit**(トークン上限)対策:
 - Rate Limit検出時、試行回数に応じた指数バックオフ待機(10秒×試行回数)を挟んで自動復帰。

第5章 実装仕様(ソースコード解説)

5.1 動作環境・依存ライブラリ

- **OS**: macOS (Monterey / Ventura / Sonoma / Sequoia)
- **Python**: 3.x
- 外部ライブラリ: openai, Pillow (PIL)
- **OS標準機能**: osascript (AppleScript), caffeinate, subprocess

5.2 コアスクリプト(**analyze_photos.py** 全体実装)

```
Python
import os
import re
import sys
```

```

import time
import shutil
import subprocess
import base64
import io
from datetime import datetime
from PIL import Image, ExifTags
from openai import OpenAI

# =====
# ⚙️ 設定エリア
# =====
KEY_FILE_PATH = os.path.expanduser("~/openai_api_key")
ALBUM_SRC = "01_AI解析用"
ALBUM_DEST = "02_解析完了"
MODEL_NAME = "gpt-4o-mini"

TEMP_DIR = "/tmp/AI_Photos"
SAVE_DIR_BASE = os.path.expanduser("~/写真分析ノート")
LOG_DIR_BASE = os.path.expanduser("~/Googleドライブ")

REFUSAL_KEYWORDS = [
    "お応えできません", "申し訳ありませんが", "リクエストに応じ",
    "I cannot fulfill", "I'm unable to", "safety system",
    "I'm sorry", "can't assist", "cannot assist", "as an ai"
]

OPENAI_API_KEY = None
if os.path.exists(KEY_FILE_PATH):
    with open(KEY_FILE_PATH, "r", encoding="utf-8") as kf:
        OPENAI_API_KEY = kf.read().strip()

if not OPENAI_API_KEY or OPENAI_API_KEY.startswith("YOUR_"):
    print("❌【エラー】OpenAI APIキーが設定されていません。")
    sys.exit(1)

try:
    subprocess.Popen(["caffeinate", "-i", "-s", "-w", str(os.getpid())])
    print("☕ Macの自動スリープ防止機能を有効化しました。")
except Exception:
    pass

if os.path.exists(TEMP_DIR):

```

```

    shutil.rmtree(TEMP_DIR)
os.makedirs(TEMP_DIR, exist_ok=True)
os.makedirs(SAVE_DIR_BASE, exist_ok=True)
os.makedirs(LOG_DIR_BASE, exist_ok=True)

client = OpenAI(api_key=OPENAI_API_KEY)

def sanitize_filename(name):
    return re.sub(r'[\\/*?:"<>|]', '_', name)

# ファイル名のみで重複判定(更新日時は無視)
def check_duplicate_and_get_path(target_dir, filename):
    clean_name = sanitize_filename(filename)
    base_save_filename = f"{clean_name}.md"
    target_path = os.path.join(target_dir, base_save_filename)

    if os.path.exists(target_dir):
        for f in os.listdir(target_dir):
            if f == base_save_filename or (f.startswith(f"{clean_name}_v") and f.endswith(".md")):
                return None, None, "SKIP"

    return target_path, base_save_filename, "NEW"

def load_and_encode_image(image_path, max_size=2048):
    with Image.open(image_path) as img:
        img_rgb = img.convert('RGB')
        img_rgb.thumbnail((max_size, max_size), Image.Resampling.LANCZOS)
        buffered = io.BytesIO()
        img_rgb.save(buffered, format="JPEG", quality=85)
        return base64.b64encode(buffered.getvalue()).decode('utf-8')

def extract_comprehensive_exif(image_path):
    exif_data = {}
    try:
        with Image.open(image_path) as img:
            raw_exif = img._getexif()
            if raw_exif:
                for tag_id, val in raw_exif.items():
                    tag_name = ExifTags.TAGS.get(tag_id, tag_id)
                    exif_data[tag_name] = val
    try:
        ifd_exif = img.getexif().get_ifd(0x8769)
        for tag_id, val in ifd_exif.items():

```

```

        tag_name = ExifTags.TAGS.get(tag_id, tag_id)
        if tag_name not in exif_data or not exif_data[tag_name]:
            exif_data[tag_name] = val
    except Exception:
        pass
except Exception as e:
    print(f" ⚠ EXIF抽出警告: {e}")
return exif_data

```

```

def append_log_safely(log_path, entry_text):
    for _ in range(3):
        try:
            with open(log_path, "a", encoding="utf-8") as lf:
                lf.write(entry_text)
            break
        except Exception:
            time.sleep(0.5)

```

AppleScript (画像・キャプション抽出)

EXPORT_WITH_CAPTION_SCRIPT = ""

on run argv

set exportFolder to item 1 of argv

set albumSrc to item 2 of argv

tell application "Photos"

set targetAlbum to null

if exists container albumSrc then

set targetAlbum to container albumSrc

else if exists album albumSrc then

set targetAlbum to album albumSrc

end if

if targetAlbum is not null then

set photoList to media items of targetAlbum

repeat with p in photoList

set fName to filename of p

set cap to description of p

if cap is missing value then set cap to ""

log "CAPTION_DATA:::" & fName & ":::" & cap

export {p} to (POSIX file exportFolder)

end repeat

end if

end tell

end run

""

```
# AppleScript (写真アプリ整理・更新)
```

```
SAFE_APPLESCRIPT = ""
```

```
on run argv
```

```
    set photoFilename to item 1 of argv
```

```
    set baseFilename to item 2 of argv
```

```
    set newTitle to item 3 of argv
```

```
    set newCaption to item 4 of argv
```

```
    set albumSrc to item 5 of argv
```

```
    set albumDest to item 6 of argv
```

```
tell application "Photos"
```

```
    if not (exists album albumDest) then
```

```
        make new album named albumDest
```

```
    end if
```

```
    set srcAlbum to null
```

```
    if exists album albumSrc then
```

```
        set srcAlbum to album albumSrc
```

```
    end if
```

```
    set destAlbum to album albumDest
```

```
    if srcAlbum is not null then
```

```
        set targetItems to (media items of srcAlbum whose filename is photoFilename)
```

```
        if (count of targetItems) is 0 then
```

```
            set targetItems to (media items of srcAlbum whose filename contains baseFilename)
```

```
        end if
```

```
        repeat with p in targetItems
```

```
            try
```

```
                set name of p to newTitle
```

```
                set description of p to newCaption
```

```
            end try
```

```
        end repeat
```

```
        if (count of targetItems) > 0 then
```

```
            try
```

```
                add targetItems to destAlbum
```

```
            end try
```

```
        end if
```

```
    end if
```

```
end tell
```

```
end run
```

```
""
```

```
print(f"📷 写真アプリの '{ALBUM_SRC}' アルバムから画像と撮影意図を抽出中...")
```

```
user_intents = {}
```

```

try:
    res = subprocess.run(["osascript", "-", TEMP_DIR, ALBUM_SRC],
input=EXPORT_WITH_CAPTION_SCRIPT, text=True, capture_output=True, timeout=300)
    for line in res.stderr.splitlines():
        if "CAPTION_DATA:::" in line:
            parts = line.split("CAPTION_DATA:::")[1].split(":::")
            if len(parts) >= 2:
                user_intents[parts[0].strip()] = parts[1].strip()
except Exception as e:
    print(f" ⚠️ キャプション読み込み警告: {e}")

```

```

files = [f for f in os.listdir(TEMP_DIR) if not f.startswith(".")]

```

```

if not files:

```

```

    print(f" ⚠️ アルバム '{ALBUM_SRC}' に対象の写真が見つかりませんでした。")
    sys.exit(0)

```

```

print(f" ✨ {len(files)} 枚の対象写真を検出しました。 \n")

```

PROMPT_TEMPLATE = """あなたは写真の美と物語を深く理解する写真美術誌のキュレーター兼写真評論家です。

提示された「写真画像」と「撮影者の意図・悩み」を踏まえ、写真の魅力を深く言語化してください。

【評価トーン指示 (50:50のゴールデンバランス)】

・**【称賛・魅力の発見 50%】**：撮影者の視点や意図、写真が持つ独自の魅力・情緒・美点を温かく肯定的に深く言語化してください。

・**【批評・成長の示唆 50%】**：プロの目から見た「撮影者の悩みへの回答」および「表現をさらに引き立てるための課題や改善策」を建設的に助言してください。

【光と強さに関するルール】

・「朝＝柔らかい光」等の固定概念を禁止します。直射光の「眩しさ」「強いシャドウ」等、現実の光の表現をありのまま評価してください。

【厳格な指示】

・「三分割法」「柔らかい光」等の定型句の使い回しを禁止します。

・「一歩前に出て広角端で寄る」「F8まで絞る」など具体的な数値・動作を指定してください。

照合用情報

- ・ファイル名: {filename}
- ・撮影日時: {photo_datetime}
- ・カメラ: {camera}
- ・レンズ: {lens}
- ・設定: {settings}
- ・撮影者の意図・悩み: {user_intent}

出力フォーマット

■TITLE: (写真のストーリーや本質を射抜くタイトルを15文字以内で出力)

■SUMMARY: (写真全体を一言で表す魅力的なサマリー・キャッチコピーを25文字以内で出力)

■SCORES:

- ・構図・構成 : [★の数5段階] (X/5)
- ・光・色彩 : [★の数5段階] (X/5)
- ・ストーリー : [★の数5段階] (X/5)
- ・技術・露出 : [★の数5段階] (X/5)
- ・独自・世界観 : [★の数5段階] (X/5)

ファイル名: {filename}

撮影日時: {photo_datetime}

ファイル更新日時: {file_mtime}

カメラ: {camera}

レンズ_焦点距離: {lens}

設定: {settings}

撮影意図: {user_intent}

【1. 情景・空気感とストーリー性】

【2. 視線誘導と構成の美学】

【3. 光の強弱・色彩と印象解析】

【4. EXIFデータの技術的役割と表現効果】

【5. 撮影者のためのステップアップ・アドバイス】

【6. フォトブック&SNSでの役割提案】

【7. 自動タグ】

#カメラ_[モデル名] #レンズ_[焦点距離] #構図_[主要な構図名] #光_[光の種別] #雰囲気_[空気感のタグ]

""

stats = {"total": len(files), "new": 0, "skipped": 0, "error": 0}

for idx, filename in enumerate(files, 1):

print(f"-----")

print(f"🔍 [{idx}/{len(files)}枚目] 検証中: {filename}")

file_path = os.path.join(TEMP_DIR, filename)

mtime_ts = os.path.getmtime(file_path)

file_mtime = datetime.fromtimestamp(mtime_ts).strftime("%Y-%m-%d %H:%M:%S")

exif_data = extract_comprehensive_exif(file_path)

```

raw_datetime = str(exif_data.get("DateTimeOriginal") or exif_data.get("DateTime") or
 "").replace('\x00', '').strip()
if raw_datetime and len(raw_datetime) >= 19:
    photo_datetime = raw_datetime[:10].replace(":", "-") + raw_datetime[10:19]
    photo_year = raw_datetime[:4] if raw_datetime[:4].isdigit() else "その他"
else:
    photo_datetime, photo_year = "不明", "その他"

year_save_dir = os.path.join(SAVE_DIR_BASE, photo_year)
os.makedirs(year_save_dir, exist_ok=True)

save_path, save_filename, status = check_duplicate_and_get_path(year_save_dir, filename)
log_file_path = os.path.join(LOG_DIR_BASE, f"写真分析ログ_{photo_year}.txt")

if status == "SKIP":
    print(f" 📌 [重複スキップ] 同名ファイル『{filename}』の解析済みノートが存在します。")
    stats["skipped"] += 1
    continue

camera_full = f"{exif_data.get('Make', '')} {exif_data.get('Model', '')}".replace('\x00', '').strip() or "
不明"
f_str = f"F{float(exif_data.get('FNumber')):.1f}" if exif_data.get('FNumber') else "不明"
ss_str = f"1/{round(1/float(exif_data.get('ExposureTime')))}秒" if exif_data.get('ExposureTime')
else "不明"
iso_str = str(exif_data.get("ISO") or "不明")
focal_str = f"{float(exif_data.get('FocalLength')):.0f}mm" if exif_data.get('FocalLength') else "不
明"
ev_str = f"{float(exif_data.get('ExposureBiasValue')):.1f} EV" if
exif_data.get('ExposureBiasValue') is not None else "±0 EV"
settings_full = f"F値: {f_str} / SS: {ss_str} / ISO: {iso_str} / 焦点距離: {focal_str} / 露出補正:
{ev_str}"

intent_text = user_intents.get(filename, "").strip() or "特になし (Allにおまかせ)"
base64_img = load_and_encode_image(file_path)
prompt = PROMPT_TEMPLATE.format(
    filename=filename, photo_datetime=photo_datetime, file_mtime=file_mtime,
    camera=camera_full, lens=str(exif_data.get("LensModel", "不明")),
    settings=settings_full, user_intent=intent_text
)

result_text, is_refused, max_retries = None, False, 2
for attempt in range(max_retries + 1):

```

```

is_refused, result_text = False, None
try:
    response = client.chat.completions.create(
        model=MODEL_NAME,
        messages=[{"role": "user", "content": [{"type": "text", "text": prompt}, {"type": "image_url",
"image_url": {"url": f"data:image/jpeg;base64,{base64_img}", "detail": "high"}}]},
        max_tokens=2500, timeout=90.0
    )
    result_text = response.choices[0].message.content
except Exception as e:
    if "429" in str(e) or "rate_limit" in str(e).lower():
        time.sleep(10 * (attempt + 1))

if result_text:
    for kw in REFUSAL_KEYWORDS:
        if kw.lower() in result_text.lower():
            is_refused = True
            break

if result_text and not is_refused:
    break
elif attempt < max_retries:
    time.sleep(3)

if not result_text or is_refused:
    append_log_safely(log_file_path, f"\n\n解析日時: {datetime.now()} | ファイル: {filename} | 【スキップ】\n")
    stats["error"] += 1
    continue

# 保存処理
with open(save_path, "w", encoding="utf-8") as f:
    f.write(result_text)
print(f"  [1/3] ノート保存成功: {photo_year}/{save_filename}")
stats["new"] += 1

log_entry = f"\n\n===== \n  解析日時: {datetime.now()} | ノート名: {save_filename} \n===== \n{result_text} \n"
append_log_safely(log_file_path, log_entry)

title_match = re.search(r"■\s*TITLE\s*[:.]\s*(.+)", result_text, re.IGNORECASE)

```

```

title_line = title_match.group(1).strip().replace("*", "") if title_match else "タイトル不明"

try:
    subprocess.run(["osascript", "-", filename, os.path.splitext(filename)[0], title_line, result_text,
ALBUM_SRC, ALBUM_DEST], input=SAFE_APPLESCRIPT, text=True, check=True, timeout=60)
    print(f" 🟢 [3/3] 写真アプリ書き込み & 『{ALBUM_DEST}』へ追加完了")
except Exception as e:
    print(f" ⚠️ 書き込み警告: {e}")

if os.path.exists(TEMP_DIR):
    shutil.rmtree(TEMP_DIR)
print("🎉 処理完遂サマリー:", stats)

```

第6章 次フェーズへの拡張ロードマップ（類似開発・サービス化ガイド）

将来的なWebサービス化・LINE公式アカウント連携、および他ツールでの再利用に向けた設計指針です。

6.1 ローカル環境依存の脱却（Web / LINE API化）

- ボトルネック: AppleScriptによるMac「写真」アプリ依存。
- 移行構成:
 - フロントエンド: Webフォーム (Next.js / HTML5) または LINE Bot (Messaging API)。
 - バックエンド: Python (FastAPI / Flask)。マルチパートで「画像ファイル+テキスト(意図)」を受け取り、OpenAI APIへリレー。

6.2 講評カード画像の自動生成（Pillow動的描画）

- 目的: SNS(X, Instagram)でシェア可能なビジュアルカードの動的作成。
- 処理フロー:
 1. 写真画像を正方形または16:9に背景ボカシ付きで配置。
 2. 生成された ■TITLE, ■SUMMARY, ■SCORES を Pillow (ImageDraw) でテキスト描画。
 3. 出力したJPEG画像をユーザーへ返信・ダウンロード提供。

このドキュメントを保存しておくことで、今後の機能拡張時や、別のLLMマルチモーダル開発を行う際の完全な「設計・実装リファレンス」としてご活用いただけます。